

SKILL PROGRAMS 11-24

11. Command History

Code

```
GNU nano 2.7.1 cmdhistory.c *
#include <stdio.h>
#include <string.h>

#define MAX 5
#define SIZE 100

int main() {
    char history[MAX][SIZE];
    int count = 0;
    char command[SIZE];

    printf("Command History System\n");

    while (count < MAX) {
        printf("Enter command: ");
        fgets(command, SIZE, stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        strcpy(history[count], command);
        count++;
    }

    printf("\nCommand History:\n");

    for (int i = 0; i < count; i++)
        printf("%d  %s\n", i + 1, history[i]);

    return 0;
}
```

Output

```
spoorthi@LAPTOP:~$ nano cmdhistory.c
spoorthi@LAPTOP:~$ gcc cmdhistory.c -o cmdhistory
spoorthi@LAPTOP:~$ ./cmdhistory
Command History System
Enter command: ls
Enter command: pwd
Enter command: date
Enter command: whoami
Enter command: gcc

Command History:
1  ls
2  pwd
3  date
4  whoami
5  gcc
spoorthi@LAPTOP:~$ |
```

12. Single Pipe

Code

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid;
    char message[] = "Hello from parent process";
    char buffer[100];

    pipe(pipefd);

    pid = fork();

    if (pid == 0) {
        close(pipefd[1]);
        read(pipefd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(pipefd[0]);
    } else {
        close(pipefd[0]);
        write(pipefd[1], message, sizeof(message));
        close(pipefd[1]);
        wait(NULL);
        printf("Parent: Child process completed\n");
    }

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q12.c
spoorthi@LAPTOP:~$ gcc q12.c -o q12
spoorthi@LAPTOP:~$ ./q12
Child received: Hello from parent process
Parent: Child process completed
spoorthi@LAPTOP:~$ |

```

13. Input and Output Redirection

Code

```

GNU nano 8.7.1                                q13.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char buffer[100];

    fd = open("input.txt", O_RDONLY);

    if (fd < 0) {
        printf("File not found\n");
        return 1;
    }

    dup2(fd, STDIN_FILENO);
    close(fd);

    fgets(buffer, sizeof(buffer), stdin);

    fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    dup2(fd, STDOUT_FILENO);
    close(fd);

    printf("Data read from input.txt: %s", buffer);

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q13.c
spoorthi@LAPTOP:~$ gcc q13.c -o q13
spoorthi@LAPTOP:~$ echo "Hello Ubuntu" > input.txt
spoorthi@LAPTOP:~$ ./q13
spoorthi@LAPTOP:~$ cat output.txt
Data read from input.txt: Hello Ubuntu
spoorthi@LAPTOP:~$ |

```

14. Append and Error Redirection

Code

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;

    fd = open("output.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);

    dup2(fd, STDOUT_FILENO);
    close(fd);

    printf("New data appended\n");

    fd = open("error.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    dup2(fd, STDERR_FILENO);
    close(fd);

    fprintf(stderr, "This is an error message\n");

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q14.c
spoorthi@LAPTOP:~$ gcc q14.c -o q14
spoorthi@LAPTOP:~$ ./q14
Data read from input.txt: Hello Ubuntu
New data appended
spoorthi@LAPTOP:~$ cat error.txt
This is an error message
spoorthi@LAPTOP:~$

```

15. Combined Redirection

Code

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd;

    fd = open("combined.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);

    close(fd);

    printf("Standard output message\n");
    fprintf(stderr, "Standard error message\n");

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q15.c
spoorthi@LAPTOP:~$ gcc q15.c -o q15
spoorthi@LAPTOP:~$ ./q15
spoorthi@LAPTOP:~$ cat combined.txt
Standard error message
Standard output message
spoorthi@LAPTOP:~$ |

```

16. Background Process

Code

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Background process started\n");
        sleep(5);
        printf("Background process completed\n");
    } else {
        printf("Parent: Background job started with PID %d\n", pid);
        printf("Parent: Returning to prompt immediately\n");
        wait(NULL);
    }

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q16.c
spoorthi@LAPTOP:~$ gcc q16.c -o q16
spoorthi@LAPTOP:~$ ./q16
Parent: Background job started with PID 993
Parent: Returning to prompt immediately
Background process started
Background process completed
spoorthi@LAPTOP:~$ |

```

17. List Active Jobs

Code

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();

    if (pid1 == 0) {
        sleep(5);
        return 0;
    }

    pid2 = fork();

    if (pid2 == 0) {
        sleep(3);
        return 0;
    }

    printf("Active Jobs:\n");
    printf("Job 1 PID: %d\n", pid1);
    printf("Job 2 PID: %d\n", pid2);

    wait(NULL);
    wait(NULL);

    printf("All jobs completed\n");

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q17.c
spoorthi@LAPTOP:~$ gcc q17.c -o q17
spoorthi@LAPTOP:~$ ./q17
Active Jobs:
Job 1 PID: 1040
Job 2 PID: 1041
All jobs completed
spoorthi@LAPTOP:~$ |

```

18. Resume Stopped Job

Code


```

GNU nano 8.7.1                                q18.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child process started\n");
        raise(SIGSTOP);
        printf("Child process resumed\n");
        return 0;
    }

    sleep(1);

    printf("Child process stopped\n");
    printf("Sending SIGCONT to resume child\n");

    kill(pid, SIGCONT);

    wait(NULL);

    printf("Child process completed\n");

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q18.c
spoorthi@LAPTOP:~$ gcc q18.c -o q18
spoorthi@LAPTOP:~$ ./q18
Child process started
Child process stopped
Sending SIGCONT to resume child
Child process resumed
Child process completed
spoorthi@LAPTOP:~$ |

```

19. SIGINT Signal Handling

Code

```

GNU nano 8.7.1                                q19.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_sigint(int sig) {
    printf("\nSIGINT received\n");
    printf("Interrupt handled safely\n");
}

int main() {
    signal(SIGINT, handle_sigint);

    printf("Program running...\n");
    printf("Press Ctrl+C to send SIGINT\n");

    while (1) {
        sleep(1);
    }

    return 0;
}

```

Output

```

Child process completed
spoorthi@LAPTOP:~$ nano q19.c
spoorthi@LAPTOP:~$ gcc q19.c -o q19
spoorthi@LAPTOP:~$ ./q19
Program running...
Press Ctrl+C to send SIGINT
C
C^C
SIGINT received
Interrupt handled safely

^C
SIGINT received
Interrupt handled safely

```

20. SIGTSTP and Process Group

Code

```

GNU nano 8.7.1                                q20.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_sigstsp(int sig) {
    printf("\nSIGTSTP received\n");
    printf("Process suspension request handled\n");
}

int main() {
    signal(SIGTSTP, handle_sigstsp);

    printf("Process Group ID: %d\n", getpgrp());
    printf("Program running...\n");
    printf("Press Ctrl+Z to send SIGTSTP\n");

    while (1)
        sleep(1);

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q20.c
spoorthi@LAPTOP:~$ gcc q20.c -o q20
spoorthi@LAPTOP:~$ ./q20
Process Group ID: 1278
Program running...
Press Ctrl+Z to send SIGTSTP
^Z
SIGTSTP received
Process suspension request handled

```

21. Module Integration and Error Handling

Code

```

#include <stdio.h>
#include <stdlib.h>

void module1() {
    printf("Module 1: Input processing successful\n");
}

void module2() {
    printf("Module 2: Command processing successful\n");
}

void module3() {
    printf("Module 3: Output processing successful\n");
}

int main() {
    printf("Integrated System\n");

    module1();
    module2();
    module3();

    printf("All modules executed successfully\n");

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q21.c
spoorthi@LAPTOP:~$ gcc q21.c -o q21
spoorthi@LAPTOP:~$ ./q21
Integrated System
Module 1: Input processing successful
Module 2: Command processing successful
Module 3: Output processing successful
All modules executed successfully
spoorthi@LAPTOP:~$ |

```

22. Valgrind and Test Script

Code

```
GNU nano 8.7.1 q22.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;

    ptr = malloc(5 * sizeof(int));

    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;

    printf("Memory allocated successfully\n");

    free(ptr);

    printf("Memory released successfully\n");

    return 0;
}
```

Output

```
spoorthi@LAPTOP:~$ nano q22.c
spoorthi@LAPTOP:~$ gcc q22.c -o q22
spoorthi@LAPTOP:~$ ./q22
Memory allocated successfully
Memory released successfully
spoorthi@LAPTOP:~$
```

23. Large Pipeline and Performance

Code

```

GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipe1[2];
    int pipe2[2];

    pipe(pipe1);
    pipe(pipe2);

    if (fork() == 0) {
        close(pipe1[0]);
        dup2(pipe1[1], STDOUT_FILENO);
        close(pipe1[1]);

        execlp("seq", "seq", "1", "10", NULL);
    }

    if (fork() == 0) {
        close(pipe1[1]);
        close(pipe2[0]);

        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe2[1]);

        execlp("grep", "grep", "1", NULL);
    }

    if (fork() == 0) {
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[1]);

        dup2(pipe2[0], STDIN_FILENO);
        close(pipe2[0]);

        execlp("wc", "wc", "-l", NULL);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    wait(NULL);
    wait(NULL);
    wait(NULL);

    printf("Large pipeline executed successfully\n");

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q23.c
spoorthi@LAPTOP:~$ gcc q23.c -o q23
spoorthi@LAPTOP:~$ ./q23
2
Large pipeline executed successfully
spoorthi@LAPTOP:~$ |

```

24. Source Code Organization and Demo

Code

```

#include <stdio.h>
#include <unistd.h>

void show_system() {
    printf("OSSP Project Demo\n");
    printf("-----\n");
    printf("1. Command History\n");
    printf("2. Pipe Communication\n");
    printf("3. Process Management\n");
    printf("4. Signal Handling\n");
    printf("5. Memory Management\n");
}

int main() {
    printf("Project Repository Demo\n\n");

    show_system();

    printf("\nCurrent Process ID: %d\n", getpid());
    printf("Demo completed successfully\n");

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano q24.c
spoorthi@LAPTOP:~$ gcc q24.c -o q24
spoorthi@LAPTOP:~$ ./q24
Project Repository Demo

OSSP Project Demo
-----
1. Command History
2. Pipe Communication
3. Process Management
4. Signal Handling
5. Memory Management

Current Process ID: 1650
Demo completed successfully
spoorthi@LAPTOP:~$ |

```